



Tecnicatura Universitaria en Programación

Metodología de Sistemas II

Trabajo Práctico Final: Semana 1

Docente: Ing. Prof. Laurent Silvina.

Integrantes:

- **Espinosa Ezequiel**
- **Maldonado Carlos**
- **Ocampo Iván**
- **Sanchez Mauricio**

Listado de Code Smells detectados y solucionados

- 1) **Duplicación de Código:** Las validaciones de los emprendedores estaban copiadas textualmente en `Emprendedor.java` y `Validadores.java`. Además, la lógica para saber si un producto tenía stock bajo se repetía en `Producto.java` (`hayStockBajo` y `isStockBajo`) y se volvía a escribir en `Reportes.java`. Estos códigos duplicados fueron unificados. Por otro lado en el caso de `Producto.java` se eliminó categoría ya que se puede obtener del vendedor (se supone que un vendedor vende productos de una sola categoría).
- 2) **Obsesión por los primitivos: rigidez:** Las categorías se manejaban con `String` ("comida", "ropa", "tecnología"), lo que es propenso a errores tipográficos. Se reemplazó con un `Enum`.
- 3) **Rigidez:** Por otro lado, las categorías al verse utilizadas en varios `if`, sumaban rigidez para la modificación ya que había que modificar al menos dos métodos. Se solucionó con el mismo `Enum`.
- 4) **Clase Dios:** La clase `Emprendedor.java` hacía varias cosas: guardar sus datos, auto-validarse y formatear el texto para la consola. Ahora solo gestiona su propio estado.
- 5) **Fragilidad:** Todos los atributos eran públicos por lo que el uso de alguna parte de código de `Emprendedor.java`, `Producto.java`, `Venta.java`, `GestorFeria.java` podía modificar o romper el funcionamiento correcto. Esto se podía ver en `GestorFeria.java` en `productoEncontrado.stock -= cantidad` que podía modificar el stock de un producto directamente violando el encapsulamiento y realizando una responsabilidad que le corresponde a la clase `Producto`.

Por otro lado, también podría encajar como **Fragilidad**, la utilización de listas paralelas en `registrarEmprendedorConProductos` en `GestorFeria.java`, mantener sincronizadas listas separadas es complejo, se debe confiar en que los índices de estas estén alineados, por ejemplo: el índice 2 de `nombresProductos`, debe coincidir con el 2 de precios y con el 2 de stocks. Ahora el objeto `Producto` encapsula su nombre, precio y stock; y por su parte el gestor recibe al objeto principal `Emprendedor` que ya trae su propia lista de objetos `Producto`.

Principios SOLID aplicados

- **Principio de Abierto/Cerrado (OCP - Open/Closed Principle):** Utilizando un `Enum` para `Categoria`, el sistema queda abierto a la extensión (podemos agregar "LIMPIEZA" al `Enum`) pero cerrado a la modificación (no tenemos que ir a buscar y modificar múltiples `if` (`cat.equals("...")`) repartidos por el código en validaciones y reportes).
- **Principio de Responsabilidad Única (SRP - Single Responsibility Principle):** Cada clase ahora tiene una única responsabilidad.
 - `Emprendedor` y `Producto` solo modelan la información y reglas de sus propios datos.
 - `Reportes` tiene la única responsabilidad de formatear y mostrar información.
 - `Validadores` es el único lugar donde residen las reglas de formato (como el email).

Refactorizaciones

- **Producto.java:** La clase era un simple contenedor de datos abiertos. Sus atributos eran públicos, lo que significa que cualquier otra clase (como Venta o GestorFeria) podía hacer `producto.stock = -50` y el producto no tenía forma de defenderse de ese estado inválido. Además, tenía redundancia: los métodos `hayStockBajo()` e `isStockBajo()` hacían exactamente lo mismo, y tenía atributos que no le correspondían directamente (`emprendedorId`), generando dependencias cruzadas.

Refactorización:

- Privacidad: Pasamos todos los atributos a `private`. Nadie desde afuera puede modificar ningún atributo.
 - Comportamiento propio: Agregamos el método `reducirStock(int cantidad)`. Ahora, si alguien quiere vender un producto, se lo tiene que pedir al producto. Esto permite que la clase valide si la cantidad solicitada es mayor al stock disponible antes de hacer la resta.
-
- **Emprendedor.java:** Cómo se mencionó anteriormente, podía considerarse una Clase Dios. Hacía demasiadas cosas: guardaba sus atributos, tenía un método extenso (`mostrarInfoYValidar`) que mezclaba validaciones de negocio (ver si el email tenía un "@") con lógica de presentación (armar un String con saltos de línea para la consola).

Refactorización:

- Desacoplamiento: Le quitamos la responsabilidad de imprimirse a sí misma (ahora se hace desde el `main`) y de validarse (lo hace la clase `Validadores`).
-
- **Venta.java:** Guardaba la fecha como un String. Además, en su método `registrarPagoYActualizarStock`, la venta violaba el encapsulamiento de `Producto` haciendo `p.stock = p.stock - this.cantidad`. También tenía toda la lógica matemática de descuento en el cálculo del total.

Refactorización:

- Tipos nativos: Reemplazamos el String de la fecha por `LocalDate`, lo que te permitiría a futuro ordenar ventas por mes o calcular vencimientos fácilmente usando las herramientas propias de Java.
- Delegación: La venta ya no resta el stock matemáticamente. Se hace `this.producto.reducirStock(this.cantidad)` en `registrarPago`. Delegando la responsabilidad a `Producto`.
- Métodos cortos: Extrajimos los `if` de los descuentos a un método privado pequeño (`aplicarDescuentos()`). Esto hace que el cálculo del total se lea como lenguaje natural.

- **Validadores.java y Reportes.java:** Ambas clases accedían directamente a las propiedades públicas de otras clases y tenían validaciones repetidas.

Refactorización:

- Validadores ahora es el único lugar del sistema que sabe cómo es un email o un teléfono válido. Si la regla cambia, se modifica solo Validadores.java.
 - Reportes ahora interactúa con el Gestor a través de sus getters (gestor.getVentas()). Actúa estrictamente como un consumidor de información que solo se encarga de darle un formato visual agradable, sin alterar absolutamente ningún dato.
- **Categoria.java (clase enumerada):** Las categorías eran String ("comida", "ropa"). Si en algún lugar del código alguien escribía "Comida" (con mayúscula) o "Ropa " (con un espacio al final), los if fallaban silenciosamente y el reporte de ventas salía vacío.
Al crear el Enum Categoria, convertimos un concepto abstracto en un tipo fuerte. El compilador de Java no permite ingresar una categoría que no exista en el sistema, eliminando de raíz un vector de posibles errores en tiempo de ejecución.
- **Main.java:** En lugar de pasar un String con la categoría "comida" y tres listas paralelas de arrays al método del Gestor, ahora se crea a "Ana" y sus "Empanadas", "Tortas" y "Alfajores" como objetos, y se asocian usando ana.agregarProducto().
Al registrar una venta, en lugar de pasar strings mágicos ("E001", "Empanadas"), le pasamos a la Venta el objeto Producto directamente. El compilador ahora garantiza que es imposible vender un producto que no existe.
Como las ventas ahora controlan el descuento internamente y el producto controla su propio stock mediante reducirStock(), el ciclo del bloque "Procesar ventas pendientes" invoca simplemente a v.registrarPago() y la cadena de responsabilidades fluye de manera segura.
Todo lo que era texto que la antigua clase Emprendedor imprimía hacia la consola (info += "Emprendedor: " ...) fue sacado a un método auxiliar estático al final de la clase Main. Así mantenemos nuestros modelos puramente lógicos e independientes de cómo queramos verlos en pantalla.